Intelligent Robotics

LAB2

郝祁 Qi Hao
Email: qhao@sustech.edu.cn

OBJECTIVES

01 General Concepts of ROS

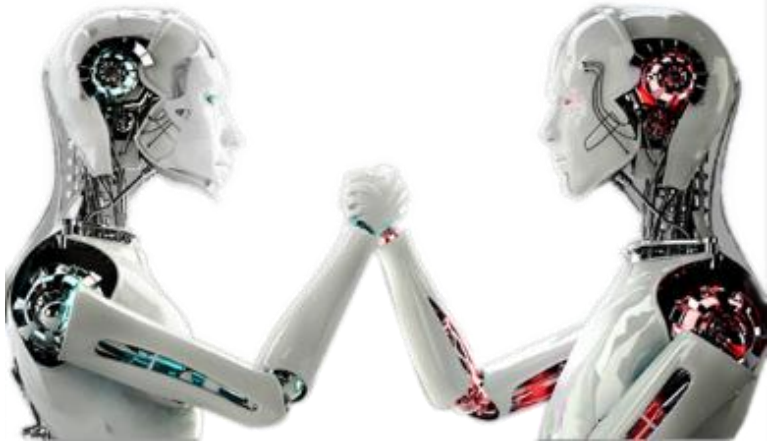
02 ROS Topic

03 ROS Service

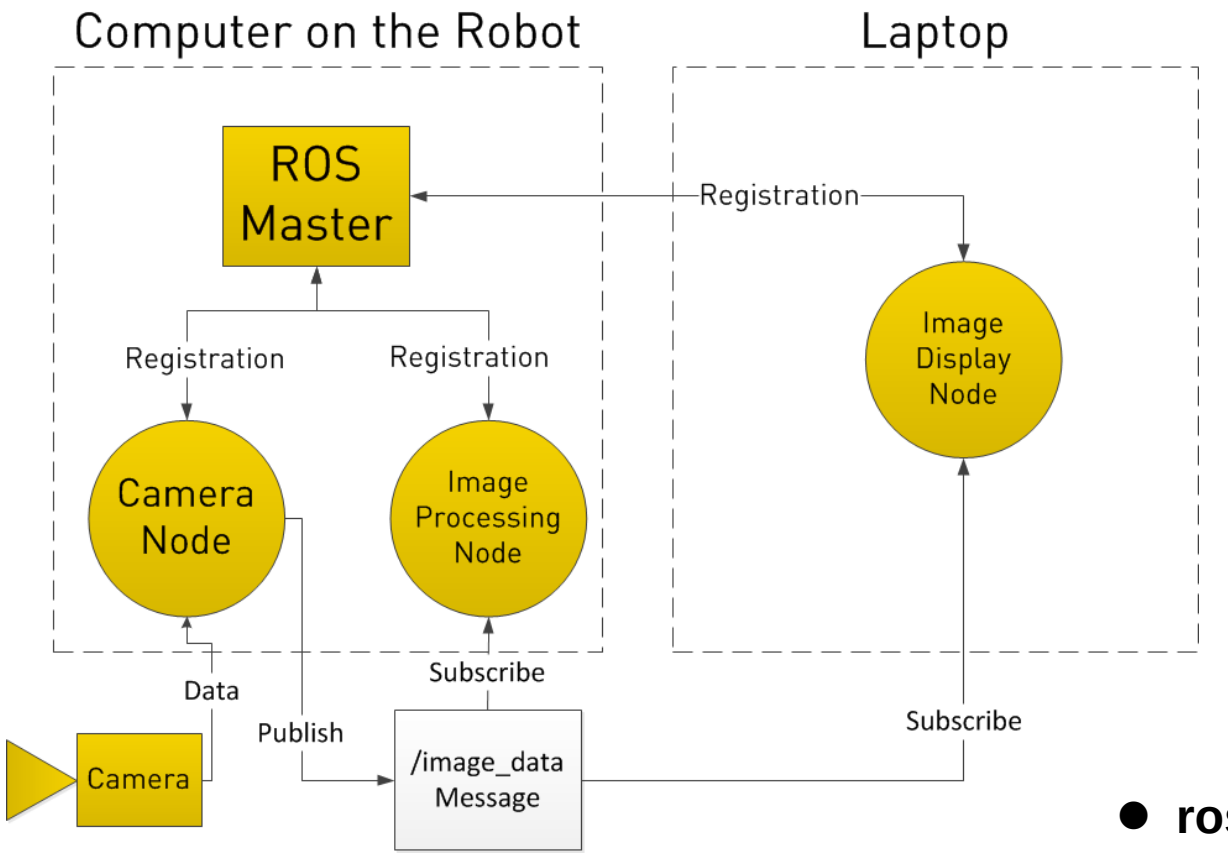
04 Parameter Server

05 roslaunch

General Concepts of ROS



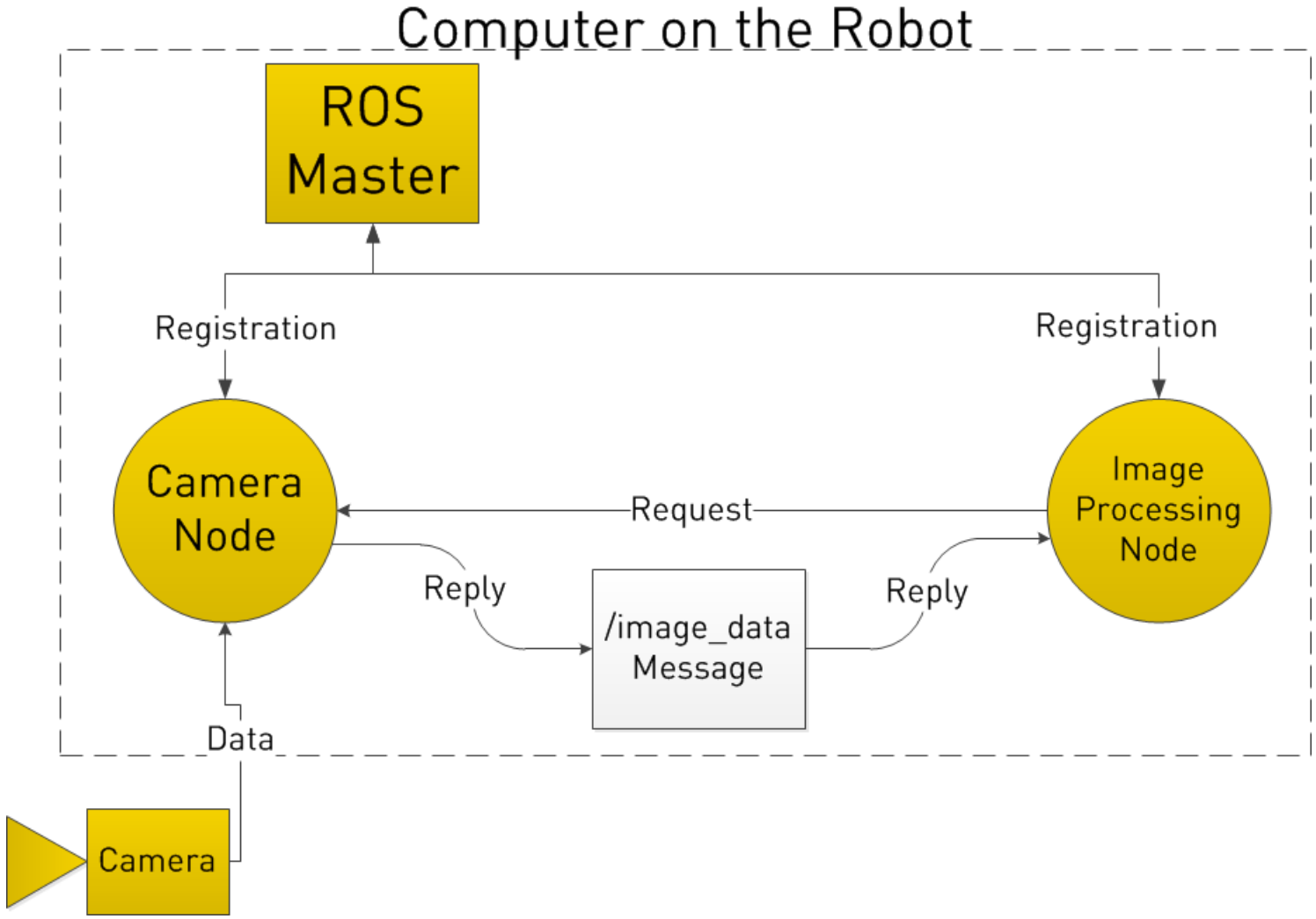
General Concepts of ROS



- **Node**: A node is an executable that uses ROS to communicate with other nodes.
- **Topic**: Nodes can publish messages to a topic as well as subscribe to a topic to receive messages.
- **Message**: ROS data type used when subscribing or publishing to a topic.
- **Master**: Name service for ROS (i.e. helps nodes find each other)

- **rosout**: ROS equivalent of stdout/stderr
- **roscore**: Master + rosout + parameter server (parameter server will be introduced later)

General Concepts of ROS



Service:Service is another way that nodes can communicate with each other. Service allows nodes to **send a request** and **receive a response**.

Commandline tools

1.Start the ROS master

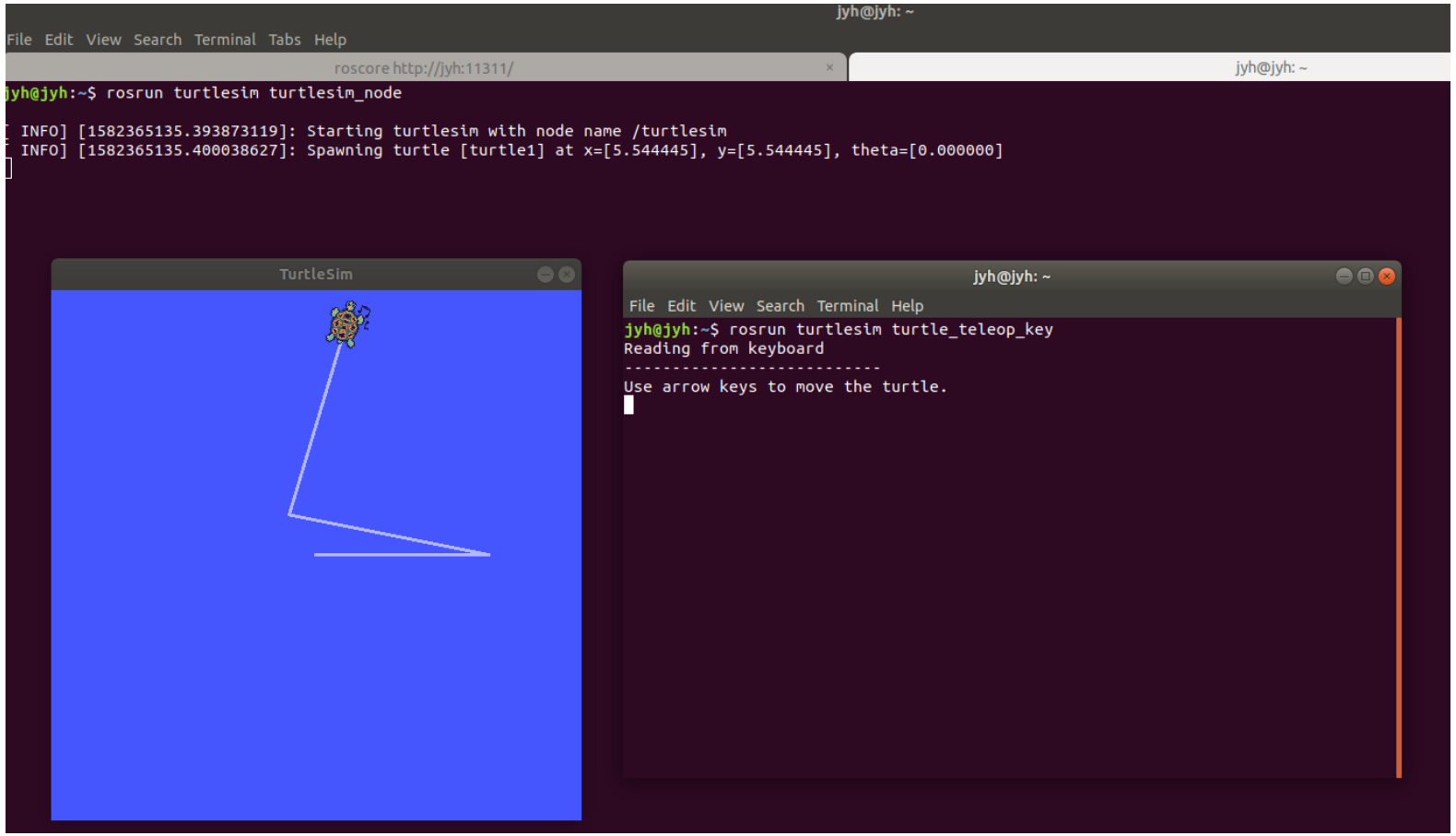
```
$roscore
```

2. Running turtle simulator

```
$rosrun turtlesim turtlesim_node
```

3.Runing control node

```
$rosrun turtlesim turtle_teleop_key
```



rosrun allows you to run a node from a given package:

```
$rosrun  [package_name] [node_name]
```

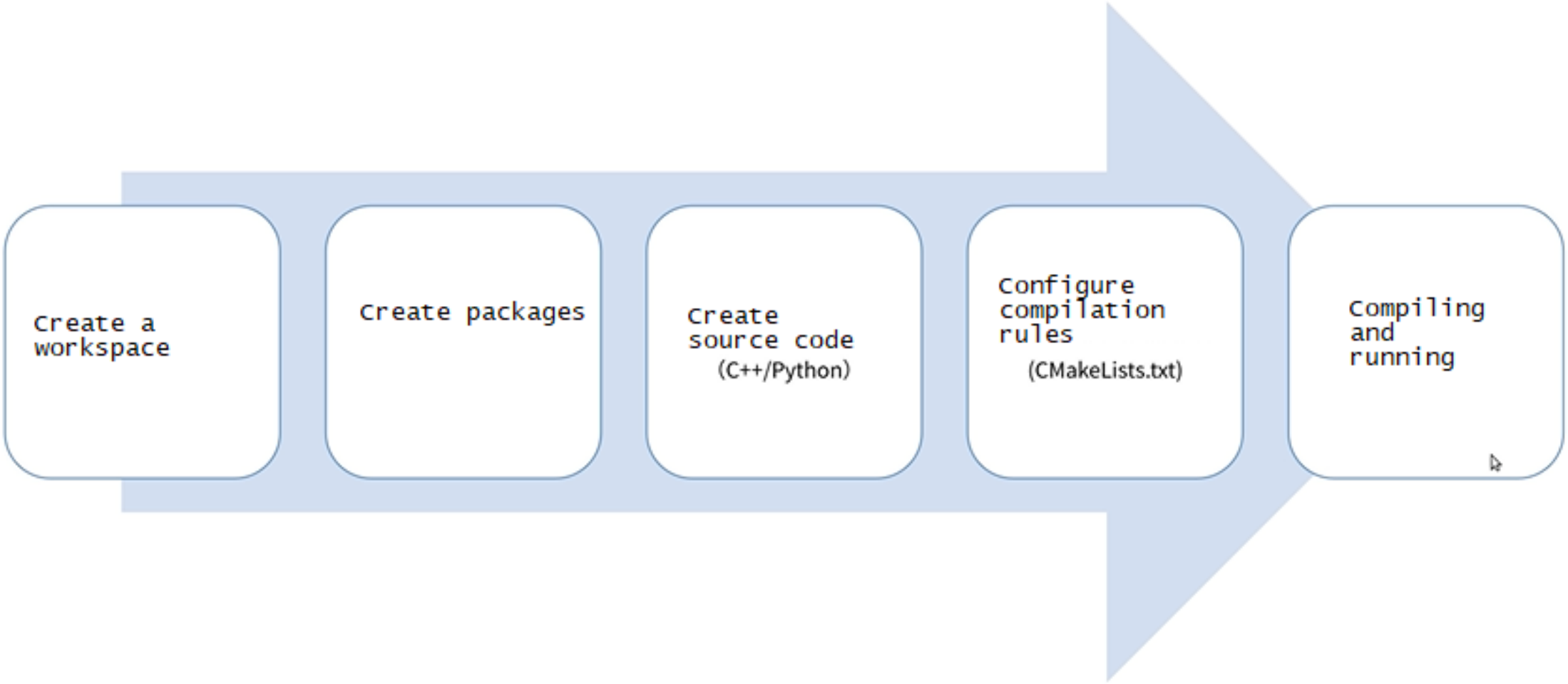
Commandline tools

- *rqt_graph*
- rosnode list
- rosnode info
- rostopic list
- rostopic type
- rosmsg show
- rostopic pub
- rostopic hz /turtle1/pose
- *rqt_plot*
- rosservice list
- rosservice call
- rosbag record -a -O *cmd_record*
- rosbag play *cmd_record.bag*

```
rostopic pub -r 10 /turtle1/cmd_vel
geometry_msgs/Twist "linear:
  x: 1.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 1.0"
```

```
rosservice call /spawn
"x: 5.0
y: 5.0
theta: 0.0
name: 'turtle2'"
```

ROS Programming



ROS Workspace

```
workspace_folder/           -- WORKSPACE
    src/                    -- SOURCE SPACE
        CMakeLists.txt      -- The 'toplevel' CMake file
        package_1/
            CMakeLists.txt
            package.xml
            ...
        package_n/
            CATKIN_IGNORE   -- Optional empty file to exclude package_n from being processed
            CMakeLists.txt
            package.xml
            ...
    build/                  -- BUILD SPACE
        CATKIN_IGNORE       -- Keeps catkin from walking this directory
    devel/                  -- DEVELOPMENT SPACE (set by CATKIN_DEVEL_PREFIX)
        bin/
        etc/
        include/
        lib/
        share/
        .catkin
        env.bash
        setup.bash
        setup.sh
        ...
    install/                -- INSTALL SPACE (set by CMAKE_INSTALL_PREFIX)
        bin/
        etc/
        include/
        lib/
        share/
        .catkin
        env.bash
        setup.bash
        setup.sh
        ...
```

- src: Source Space
- build: Build space
- devel：Development Space
- install: Install Space

step1 . Create and build a catkin workspace

```
$mkdir -p ~/catkin_ws/src
$cd ~/catkin_ws/src
$catkin_init_workspace
$cd ~/catkin_ws
$catkin_make
```

step 2.set env

```
$source devel/setup.bash
```

step 3.check env

```
$echo $ROS_PACKAGE_PATH
```

ROS Package

1. What are ROS Packages

Packages are the software organization unit of ROS code. Each package can contain **libraries**, **executables**, **scripts**, or **other artifacts**.

```
workspace_folder/          -- WORKSPACE
  src/                     -- SOURCE SPACE
    CMakeLists.txt         -- 'Toplevel' CMake file, provided by catkin
    package_1/
      CMakeLists.txt       -- CMakeLists.txt file for package_1
      package.xml          -- Package manifest for package_1
    ...
    package_n/
      CMakeLists.txt       -- CMakeLists.txt file for package_n
      package.xml          -- Package manifest for package_n
```

ROS Package

2. The package must contain:

- A catkin compliant **package.xml** file
- A **CMakeLists.txt** which uses catkin.

3. Practice Creating a ROS Package

```
$catkin_create_pkg <package_name> [depend1] [depend2] [depend3]
```

step 1.Creating a catkin Package

```
$cd ~/catkin_ws/src
$catkin_create_pkg beginner_tutorials std_msgs rospy roscpp
```

step 2.Building a catkin workspace

```
$cd ~/catkin_ws
$catkin_make
```

step 3.Sourcing the setup file

```
$source devel/setup.bash
```

4. Practice the following ROS tools

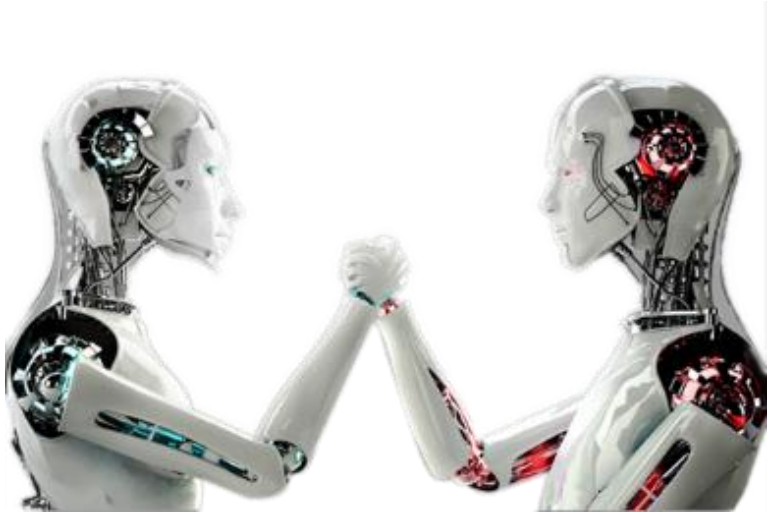
```
$ rospack depends1 beginner_tutorials
$ rospack find beginner_tutorials
$ roscd beginner_tutorials
$ roscd log
$ rosls beginner_tutorials
```

5. Learn the structure of package.xml and CMakeLists.txt

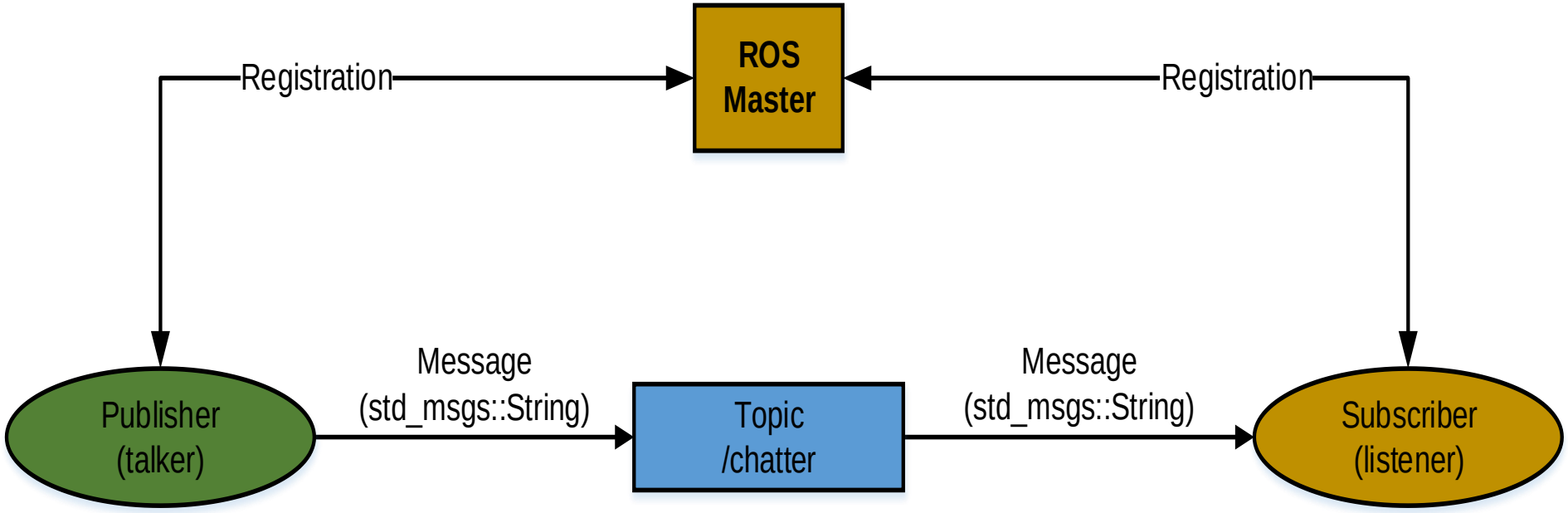
NOTE:

Under the same workspace, packages with the same name are not allowed.
under different workspaces, packages with the same name are allowed.

ROS Topic



ROS Topic


ROS
Master
Registration
Registration
Publisher
(talker)
Message
(std_msgs::String)
Topic
/chatter
Message
(std_msgs::String)
Subscriber
(listener)

ROS Topic :Writing a Simple Publisher

src > beginner_tutorials > src > talker.cpp

```cpp
#include "std_msgs/String.h"
#include <sstream>
/**
 * This tutorial demonstrates simple sending of messages over the ROS system.
 */
int main(int argc, char **argv)
{
  ros::init(argc, argv, "talker");

  ros::NodeHandle n;

  ros::Publisher chatter_pub = n.advertise<std_msgs::String>("chatter", 1000);

  ros::Rate loop_rate(10);
  int count = 0;
  while (ros::ok())
  {

    // This is a message object. You stuff it with data, and then publish it.
    std_msgs::String msg;

    std::stringstream ss;
    ss << "hello world " << count;
    msg.data = ss.str();

    ROS_INFO("%s", msg.data.c_str());
    chatter_pub.publish(msg);

    //ros::spinOnce();
    loop_rate.sleep();
    ++count;
  return 0;
}
```

1. How to Write a Publisher

Step 1. Initialize the ROS system
Step 2. Register node information with the ROS Master
Step 3. Create a message
Step4. Loop while publishing messages to Topic

2. Building the Publisher node

Add two lines to the bottom of the **CMakeLists.txt:**

```
add_executable(talker src/talker.cpp)
target_link_libraries(talker ${catkin_LIBRARIES})
```

ROS Topic :Writing a Simple Subscriber

src > beginner_tutorials > src > listener.cpp > main(int, char **)

```cpp
#include "ros/ros.h"
#include "std_msgs/String.h"

/**
 * This tutorial demonstrates simple receipt of messages over the ROS system.
 */
void chatterCallback(const std_msgs::String::ConstPtr& msg)
{
  ROS_INFO("I heard: [%s]", msg->data.c_str());
}

int main(int argc, char **argv)
{

  ros::init(argc, argv, "listener");


  ros::NodeHandle n;


  ros::Subscriber sub = n.subscribe("chatter", 1000, chatterCallback);


  ros::spin();

  return 0;
}
```

1. How to Write a Subscriber

Step 1. Initialize the ROS system
Step 2. Subscribe to the Topic
Step 3. Spin, waiting for messages to arrive
Step4. When a message arrives, the Calback function is called

2. Building the Subscriber node

Add two lines to the bottom of the **CMakeLists.txt:**

```
add_executable(listener src/listener.cpp)
target_link_libraries(listener  ${catkin_LIBRARIES})
```

ROS Topic

3. Examine the simple publisher

1.Start the ROS master

```
$roscore
```

2.Running the Publisher

```
$rosrun beginner_tutorials talker
```

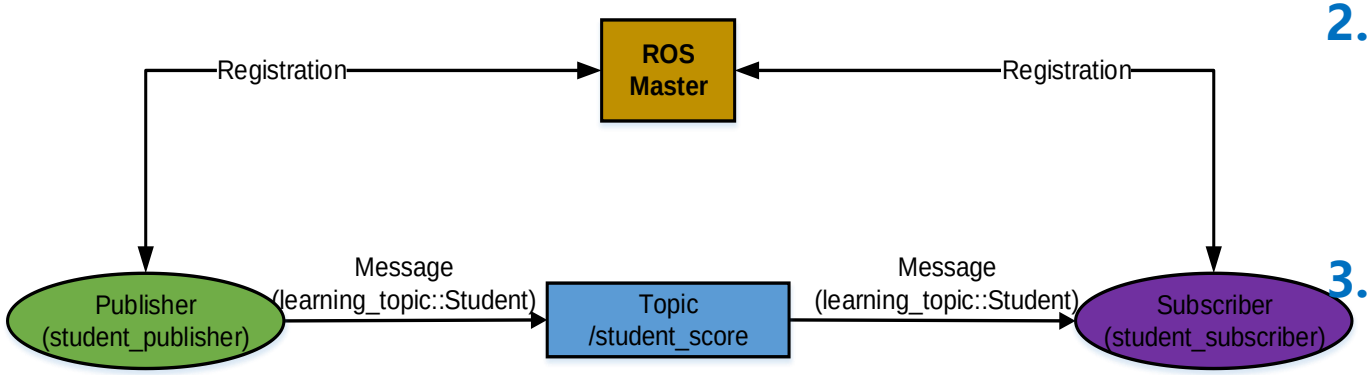
```
INFO] [1610811411.283346123]: hello world 218
INFO] [1610811411.383177625]: hello world 219
INFO] [1610811411.484056539]: hello world 220
INFO] [1610811411.582578680]: hello world 221
INFO] [1610811411.683299069]: hello world 222
INFO] [1610811411.782671216]: hello world 223
INFO] [1610811411.883105254]: hello world 224
INFO] [1610811411.983029624]: hello world 225
INFO] [1610811412.082568568]: hello world 226
INFO] [1610811412.182713893]: hello world 227
INFO] [1610811412.283092786]: hello world 228
INFO] [1610811412.383011922]: hello world 229
INFO] [1610811412.483599374]: hello world 230
INFO] [1610811412.582485916]: hello world 231
INFO] [1610811412.682987757]: hello world 232
INFO] [1610811412.782710016]: hello world 233
INFO] [1610811412.883114607]: hello world 234
INFO] [1610811412.983183557]: hello world 235
INFO] [1610811413.082472649]: hello world 236
INFO] [1610811413.182770237]: hello world 237
INFO] [1610811413.282425500]: hello world 238
INFO] [1610811413.383153170]: hello world 239
INFO] [1610811413.483117763]: hello world 240
```

3.Running the Subscriber

```
$rosrun beginner_tutorials  listener
```

```
INFO] [1610811393.082673747]: I heard: [hello world 36]
INFO] [1610811393.183529274]: I heard: [hello world 37]
INFO] [1610811393.283159031]: I heard: [hello world 38]
INFO] [1610811393.382973276]: I heard: [hello world 39]
INFO] [1610811393.483036596]: I heard: [hello world 40]
INFO] [1610811393.582793369]: I heard: [hello world 41]
INFO] [1610811393.683695139]: I heard: [hello world 42]
INFO] [1610811393.783002337]: I heard: [hello world 43]
INFO] [1610811393.882776961]: I heard: [hello world 44]
INFO] [1610811393.984102399]: I heard: [hello world 45]
INFO] [1610811394.083575950]: I heard: [hello world 46]
INFO] [1610811394.183728469]: I heard: [hello world 47]
INFO] [1610811394.283490633]: I heard: [hello world 48]
INFO] [1610811394.384312610]: I heard: [hello world 49]
INFO] [1610811394.482698160]: I heard: [hello world 50]
INFO] [1610811394.583539003]: I heard: [hello world 51]
INFO] [1610811394.685269004]: I heard: [hello world 52]
INFO] [1610811394.783013789]: I heard: [hello world 53]
INFO] [1610811394.882894785]: I heard: [hello world 54]
INFO] [1610811394.983724505]: I heard: [hello world 55]
INFO] [1610811395.083366840]: I heard: [hello world 56]
INFO] [1610811395.182944370]: I heard: [hello world 57]
INFO] [1610811395.284215594]: I heard: [hello world 58]
```

ROS msg



The field types for msg:

- **int8, int16, int32, int64** (plus uint*)
- **float32, float64**
- **string**
- **time**, duration
- **other msg files**
- variable-length **array[]** and fixed-length **array[C]**

1. Define a msg file

Student.msg

```
string name
string id
uint8 grade
float32 score
uint8 freshman = 1
uint8 sophomore =2
uint8 junior =3
uint8 Senior =4
```

2. Add package dependencies in **package.xml**

```
<build_depend>message_generation</build_depend>
<exec_depend>message_runtime</exec_depend>
```

3. Add build options in **cmakelists.txt**

① **find_package**(... message_generation)
② **add_message_files**(FILES Student.msg)
③ **generate_messages**(DEPENDENCIES std_msgs)
④ **catkin_package**(... message_runtime)

4.Building

```
$cd ~/catkin_ws
$catkin_make
```

5.Checking msg

```
$rosmsg show beginner_tutorials/Student
```

6.Using msg

ROS msg : Publisher

src > beginner_tutorials > src > student_publisher.cpp > main(int, char **)

```cpp
#include <ros/ros.h>
#include "learning_topic/Student.h"

int main(int argc, char **argv)
{
    // ROS init
    ros::init(argc, argv, "student_publisher");

    // Create a node handle
    ros::NodeHandle n;

    //  Create a Publisher, publish a topic named /student, message type:learning_topic::Student, queue length 10
    ros::Publisher person_info_pub = n.advertise<learning_topic::Student>("/student_score", 10);

    // Set the frequency of the loop
    ros::Rate loop_rate(1);

    int count = 0;
    while (ros::ok())
    {
        // Initialize the message of learning_topic::Student
        learning_topic::Student msg;
        msg.name = "Jimmy";
        msg.id = "12345678";
        msg.grade  = learning_topic::Student::junior;
        msg.score  = 90.8;

        // publish message
        person_info_pub.publish(msg);

        ROS_INFO("Publish Student's score : name:%s  id:%s grade:%d  float32:%.2f",
                 msg.name.c_str(),msg.id.c_str(), msg.grade, msg.score);
        // Delay by loop frequency
        loop_rate.sleep();
    }

    return 0;
}
```

ROS msg: Subscriber

```
#include <ros/ros.h>
#include "learning_topic/Student.h"


//After receiving the subscribed message,
// enter the message callback function
void studentCallback(const learning_topic::Student::ConstPtr& msg)
{
    // Print out the received message
    ROS_INFO("Subcribe Student's score : name:%s  id:%s grade:%d  score:%.2f",
            msg->name.c_str(),msg->id.c_str(), msg->grade, msg->score);
}

int main(int argc, char **argv)
{
    // Initialize the ROS node
    ros::init(argc, argv, "student_subscriber");

    // Create a node handle
    ros::NodeHandle n;


    // Create a Subscriber, subscribe to the topic /student,
    // and register the callback function studentCallback
    ros::Subscriber person_info_sub = n.subscribe("/student_score", 10, studentCallback);

    // Enter a loop, pumping callbacks
    ros::spin();

    return 0;
}

```

ROS msg

Add following lines to the bottom of the **CMakeLists.txt:**

```
add_executable(student_subscriber src/student_subscriber.cpp)
add_dependencies(student_subscriber beginner_tutorials_generate_messages_cpp)
target_link_libraries(student_subscriber  ${catkin_LIBRARIES})

add_executable(student_publisher src/student_publisher.cpp)
target_link_libraries(student_publisher ${catkin_LIBRARIES})
add_dependencies(student_publisher beginner_tutorials_generate_messages_cpp)
```

1.Start the ROS master

```
$roscore
```

2.Running the publisher

```
$rosrun beginner_tutorials student_publisher
```

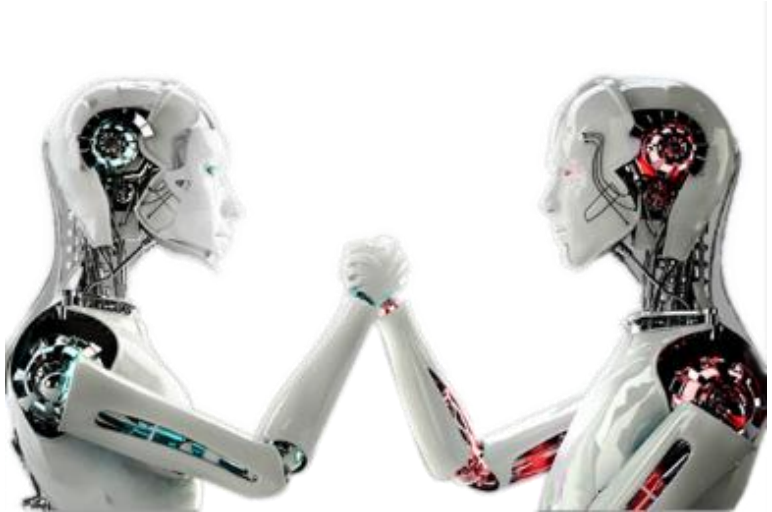
3.Running the subscriber

```
$rosrun beginner_tutorials  student_subscriber
```

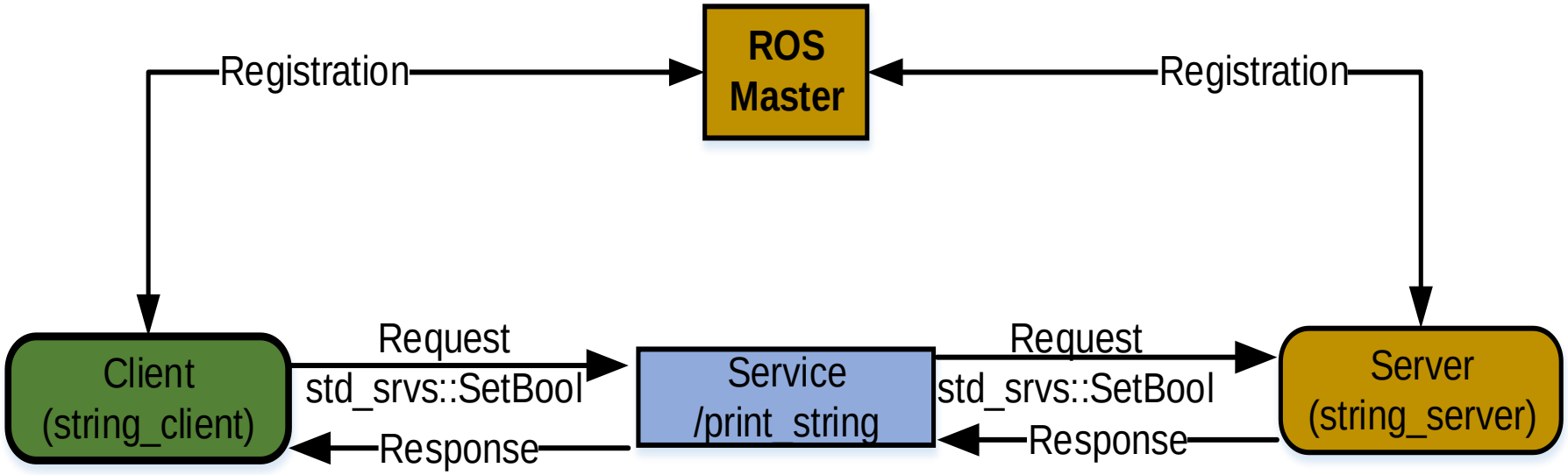
```
[ INFO] [1610811840.540058120]: Publish Student's score : name:Jimmy  id:12345678 grade:3  float32:
90.80
[ INFO] [1610811841.540819999]: Publish Student's score : name:Jimmy  id:12345678 grade:3  float32:
90.80
[ INFO] [1610811842.540129146]: Publish Student's score : name:Jimmy  id:12345678 grade:3  float32:
90.80
[ INFO] [1610811843.540877858]: Publish Student's score : name:Jimmy  id:12345678 grade:3  float32:
90.80
[ INFO] [1610811844.540276106]: Publish Student's score : name:Jimmy  id:12345678 grade:3  float32:
90.80
[ INFO] [1610811845.540533043]: Publish Student's score : name:Jimmy  id:12345678 grade:3  float32:
```

```
[ INFO] [1610811859.540690451]: Subcribe Student's score : name:Jimmy  id:12345678 grade:3  score:9
0.80
[ INFO] [1610811860.541289470]: Subcribe Student's score : name:Jimmy  id:12345678 grade:3  score:9
0.80
[ INFO] [1610811861.542558608]: Subcribe Student's score : name:Jimmy  id:12345678 grade:3  score:9
0.80
[ INFO] [1610811862.542215014]: Subcribe Student's score : name:Jimmy  id:12345678 grade:3  score:9
0.80
[ INFO] [1610811863.540879951]: Subcribe Student's score : name:Jimmy  id:12345678 grade:3  score:9
0.80
[ INFO] [1610811864.541744893]: Subcribe Student's score : name:Jimmy  id:12345678 grade:3  score:9
0.80
```

ROS Service



ROS Service


ROS
Master
Registration
Registration
Client
(string_client)
Request
std_srvs::SetBool
Response
Service
/print_string
Request
std_srvs::SetBool
Response
Server
(string_server)

Writing a Server

```
$rossrv show std_srvs/SetBool
```

```
$chmod +x server.py
```

1. Writing a Server Node

Step 1. Initialize the ROS system
Step 2. Create a Service
Step 3. Spin, waiting for request to arrive
Step4. When a request arrives, the handle function is called

2. Building server node

src > beginner_tutorials > scripts > server.py > stringCallback

Set as interpreter

```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-
# 该例程将提供print_string服务，std_srvs::SetBool

import rospy
from std_srvs.srv import SetBool, SetBoolResponse

def stringCallback(req):

    if req.data:
        rospy.loginfo("Hello ROS!")


        return SetBoolResponse(True, "Print Successully")
    else:

        return SetBoolResponse(False, "Print Failed")

def string_server():

    rospy.init_node('string_server')


    s = rospy.Service('print_string', SetBool, stringCallback)


    print "Ready to print hello string."
    rospy.spin()

if __name__ == "__main__":
    string_server()

```

Writing a Simple Client

src > beginner_tutorials > scripts > client.py > ...

```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-


import sys
import rospy
from std_srvs.srv import SetBool, SetBoolRequest

def string_client():

    rospy.init_node('string_client')


    rospy.wait_for_service('print_string')
    try:
        string_client = rospy.ServiceProxy('print_string', SetBool)


        response = string_client(True)
        return response.success, response.message
    except rospy.ServiceException, e:
        print "Service call failed: %s"%e

if __name__ == "__main__":

    print "Response : [%s] %s" %(string_client())
```

$chmod +x client.py

1. Writing a Client Node

Step 1. Initialize the ROS system
Step 2. Creates a client
Step 3. Request service data
Step4. wait for response

2. Building node

ROS Service

1.Start the ROS master

```
$roscore
```

2. Running server

```
$rosrun beginner_tutorials server.py
```

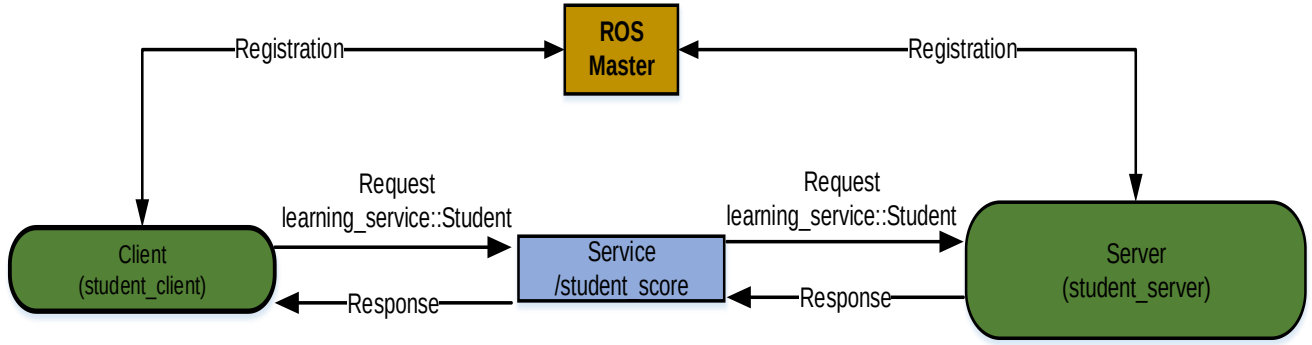
```
^Cjyh@jyh:~/catkin_ws$ rosrun beginner_tutorials server.py
Ready to print hello string.
[INFO] [1610812557.083518]: Hello ROS!
[INFO] [1610812561.322457]: Hello ROS!
[INFO] [1610812562.635151]: Hello ROS!
[INFO] [1610812563.771273]: Hello ROS!
```

3.Running client

```
$rrosrun beginner_tutorials  client.py
```

```
jyh@jyh:~/catkin_ws$ rosrun beginner_tutorials  client.py
Response : [True] Print Successully
jyh@jyh:~/catkin_ws$ rosrun beginner_tutorials  client.py
Response : [True] Print Successully
jyh@jyh:~/catkin_ws$ rosrun beginner_tutorials  client.py
Response : [True] Print Successully
jyh@jyh:~/catkin_ws$
```

ROS srv



1. Define a srv file

src > beginner_tutorials > srv > StuScore.srv

```
string id
---
string name
string id
uint8 grade
float32 score
uint8 freshman = 1
uint8 sophomore =2
uint8 junior =3
uint8 Senior =4
```

2. Add package dependencies in **package.xml**

```
<build_depend>message_generation</build_depend>
<exec_depend>message_runtime</exec_depend>
```

3. Add build options in **cmakelists.txt**

① **find_package**(... message_generation)
② add_service_files(FILES **StuScore**.srv)
③ **generate_messages**(DEPENDENCIES std_msgs)
④ **catkin_package**(... message_runtime)

4.Building srv

```
$cd ~/catkin_ws
$catkin_make
```

5.Checking srv

```
$rossrv show beginner_tutorials/StuScore
```

6.Using srv

ROS srv

```python
import rospy
# from  beginner_tutorials.srv import StuScore, StuScoreResponse
from  beginner_tutorials.srv import *
def studentCallback(req):
    # Display request data
    rospy.loginfo("the student id:%s ", req.id)


    # Feedback data
    return StuScoreResponse("Jimmy", "12345678",StuScoreResponse.junior,95.8)

def student_server():
    # Initialize the ROS nodes
    rospy.init_node('student_server')

    # Create a server named /show_student and
    # register the callback function studentCallback
    s = rospy.Service('/student_score', StuScore, studentCallback)

    # Enter the event loop and
    # wait for the callback function studentCallback
    print "Ready to response student's score."
    rospy.spin()

if __name__ == "__main__":
    student_server()
```

ROS srv

```python
import sys
import rospy
from  beginner_tutorials.srv import *
def student_client():
    # Initialize the ROS nodes
    rospy.init_node('student_client')

    # Wait for the /student_score service to start
    rospy.wait_for_service('/student_score')
    try:
        # Create a service client, and connect to the service named /student_score
        student_client = rospy.ServiceProxy('/student_score', StuScore)

        # Request service , input request data
        response = student_client("12345678")
        return response
    except rospy.ServiceException, e:
        print "Service call failed: %s"%e

if __name__ == "__main__":
    #The service is invoked and the result is displayed
    print "Show Student's score:\n%s" %(student_client())
```

ROS srv

- **Building Publisher node**

- **Running**

```
jyh@jyh:~/catkin_ws$ roscore
```

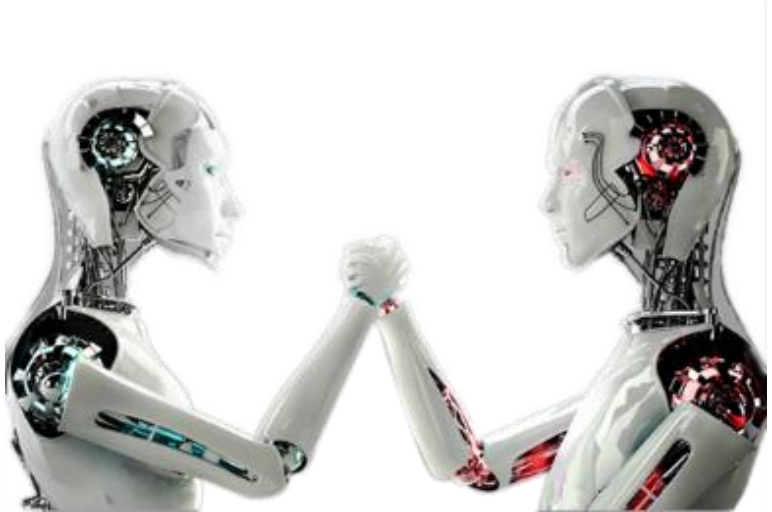
```
jyh@jyh:~/catkin_ws$ rosrun beginner_tutorials student_server.py
```

```
jyh@jyh:~/catkin_ws$ rosrun beginner_tutorials student_client.py
```

```
[ INFO] [1610812913.699089567]: Ready to response student's score.
[ INFO] [1610812944.922166517]: Student: id: 12345678
[ INFO] [1610812955.827368201]: Student: id: 12345678
[ INFO] [1610812957.065851587]: Student: id: 12345678
```

```
INFO] [1610812944.914598699]: Call service to show student's score[ id:12345678 ]
INFO] [1610812944.922343710]: Show student's score[name:Jimmy, id:12345678, grade:2, score:90.50]
```

Parameter Server

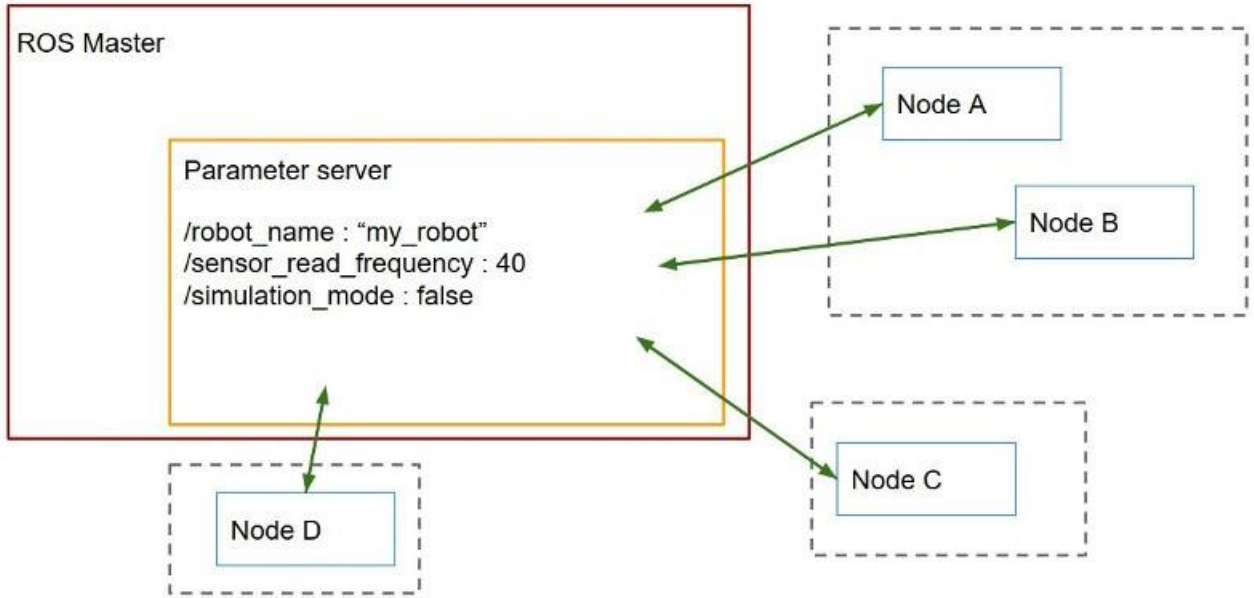


Parameter Server

roscore : master + **rosout** (stdout/stderr) + **parameter server**

➢ What is a parameter server

The parameter server is basically a dictionary containing global variables which are accessible from anywhere in the current ROS environment. **Those global variables are called ROS parameters.**



ROS parameter model

A ROS parameter has a **name**, and a **data type**. Among the most common types, you can use:

- Boolean
- Integer number
- Double number
- String
- List of previous data types

➢ YAML parameter file

```
background_b: 122
background_g: 86
background_r: 69
rosdistro: 'melodic

  '
roslaunch:
  uris: {host_jyh__33061: 'http://jyh:33061/'}
rosversion: '1.14.3

  '
run_id: f1cb9a04-593b-11ea-97c0-000c292c57f8
```

➢ rosparam command-line tool

- rosparam list
- rosparam get *param_name*
- rosparam set *param_name param_value*
- rosparam dump file_name.yaml
- rosparam load file_name.yaml
- rosparam delete *param_name*

Parameter Server

➢ Programming for ROS parameters

1. Prerequisites

```
$cd ~/catkin_ws/src
$catkin_create_pkg learning_parameter rospy roscpp
std_msgs geometry_msgs turtlesim
```

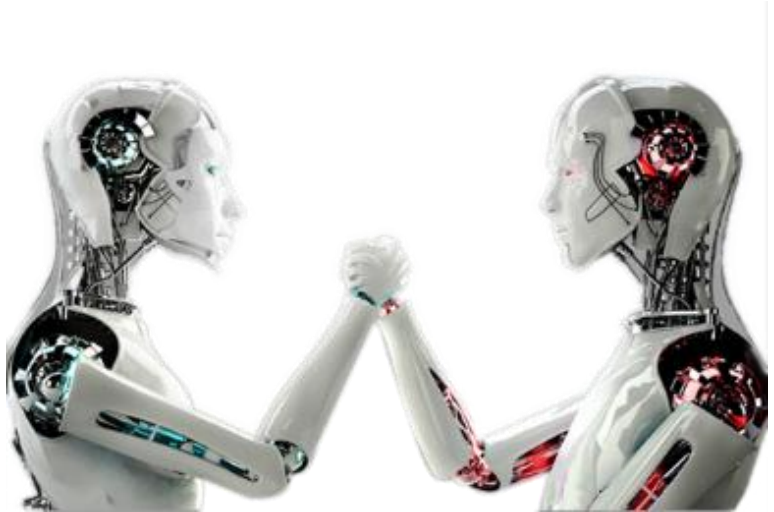
2. Writing a parameter config node

3. Building and Running

```
$cd ~/catkin_ws
$catkin_make
$rosrun learning_parameter parameter_config.py
```

```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-
# This routine sets/reads the parameters in the turtle routine

import sys
import rospy
from std_srvs.srv import Empty


def parameter_config():
    # ROS node initialization
    rospy.init_node('parameter_config', anonymous=True)

    # Read the background color parameter
    red = rospy.get_param('/turtlesim/background_r')
    green = rospy.get_param('/turtlesim/background_g')
    blue = rospy.get_param('/turtlesim/background_b')

    rospy.loginfo("Get Backgroud Color[%d, %d, %d]", red, green, blue)

    # Set background color parameters
    rospy.set_param("/turtlesim/background_r", 255);
    rospy.set_param("/turtlesim/background_g", 255);
    rospy.set_param("/turtlesim/background_b", 255);

    rospy.loginfo("Set Backgroud Color[255, 255, 255]");

    # Read the background color parameter
    red = rospy.get_param('/turtlesim/background_r')
    green = rospy.get_param('/turtlesim/background_g')
    blue = rospy.get_param('/turtlesim/background_b')

    rospy.loginfo("Get Backgroud Color[%d, %d, %d]", red, green, blue)

    # Call the service and refresh the background color
    rospy.wait_for_service('/clear')
    try:
        clear_background = rospy.ServiceProxy('/clear', Empty)

        # Request service
        response = clear_background()
        return response
    except rospy.ServiceException, e:
        print "Service call failed: %s" % e


if __name__ == "__main__":
    parameter_config()
```

Roslaunch



roslaunch

- **roslauch** is a tool for launching multiple nodes (as well as setting parameters) , including Master if not running.
- **roslaunch** starts nodes as defined in **a launch file**.
- The **launch file** USES an XML file to configure and launch multiple nodes

```
<launch>
  <group ns="turtlesim1">
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
  </group>

  <group ns="turtlesim2">
    <node pkg="turtlesim" name="sim" type="turtlesim_node"/>
  </group>

  <node pkg="turtlesim" name="mimic" type="mimic">
    <remap from="input" to="turtlesim1/turtle1"/>
    <remap from="output" to="turtlesim2/turtle1"/>
  </node>

</launch>
```

➢ roslaunch command-line tool

Usage:

- $ roslaunch [package] [filename.launch]

roslaunch

➢ The syntax of the launch file

Tag	Description	Usage
<launch>	the root element of any roslaunch file	
<node>	Launch a node.	<node pkg="pkg-name" type="executable-name" name="node-name" args="arg1 arg2 arg3" respawn="true" output="log|screen" /> ● pkg: Package of node. ● type:Node type. ● name:Node name. ● args,output, respawn,ns....
<param>	Set a parameter on the Parameter Server	<param name="arg-name" type="arg-type" value="arg-value" /> ● name: Parameter name. ● value:Defines the value of the parameter. ● type:**str|int|double|bool|yaml** , If you don't specify the type, roslaunch will attempt to automatically determine the type.
<rosparam>	Set ROS parameters for the launch using a rosparam file.	<rosparam command="load|dump|delete" file="$(find pkg-name)/path/params.yaml" /> ● command: rosparam command ● file: Name of rosparam file ● param,ns ...

roslaunch

➢ The syntax of the launch file

Tag	Description	Usage
<remap>	Declare a name remapping.	<remap from="original-name" to="new-namel"/> ● <remap from="/turtlebot/cmd_vel" to="/cmd_vel"/>
<include>	Include other roslaunch files.	<include file = "$(find pkg-name)/path/ohter.launch">
<arg>	Declare an argument.	<arg name="arg-name" value="arg-value" /> ● <param name="arg-name" value="$(arg arg-value)"/> ● <node pkg="pkg-name" type="executable-name" name="node-name" args="$(arg arg-name)" />
<group>	Group enclosed elements sharing a namespace or remap.	ns="namespace" (optional) ns: Assign the group of nodes to the specified namespace.

More info

http://wiki.ros.org/roslaunch/XML

http://wiki.ros.org/roslaunch/Tutorials/Roslaunch%20tips%20for%20larger%20projects

roslaunch

➢ launch demo1

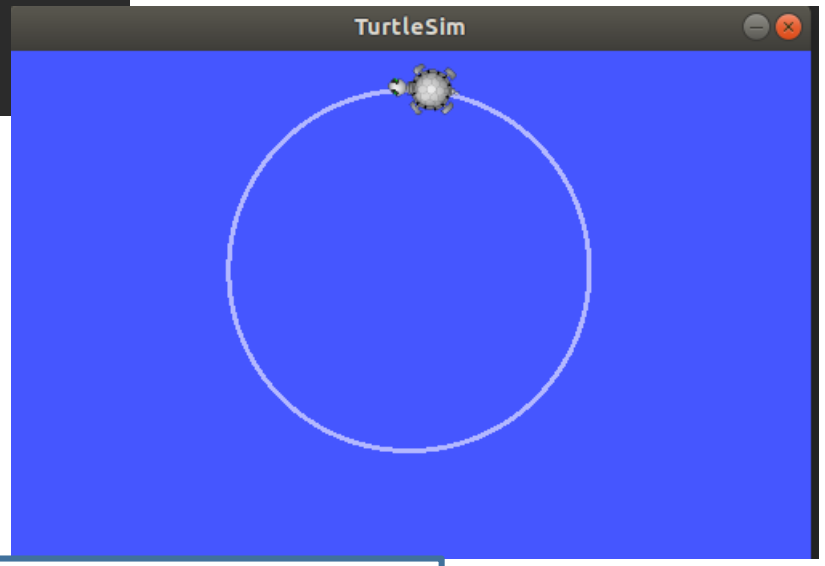
1. turtlesim_parameter_config.launch & param.yaml

turtlesim_parameter_config.launch

```
<launch>
    <param name="/number"   value="3"/>
    <node pkg="learning_launch" type="velocity_publisher.py" name="publisher" output="screen" >
            <param name="publish_frequency" type="double" value="10.0" />
            <rosparam file="$(find learning_launch)/config/param.yaml" command="load"/>
    </node>

    <node pkg="learning_launch" type="pose_subscriber.py" name="subscriber" output="screen" />
    <node pkg="turtlesim" type="turtlesim_node" name="turtle_1" output="screen"  />

</launch>
```

param.yaml

```
linear_x: 1.5
group:
  angular_z: 1.2
```



2. Modify the code

```
publish_frequency = rospy.get_param('~publish_frequency')
#Set the frequency of the loop
rate = rospy.Rate(publish_frequency)
```

```
vel_msg.linear.x = rospy.get_param('~linear_x')
vel_msg.angular.z = rospy.get_param('~group/angular_z')
```

3. Run launch file using roslaunch

```
$roslaunch learning_launch turtlesim_parameter_config.launch
```

```
$ros_graph
```

roslaunch

launch demo2

1. turtlesim_remap_include.launch

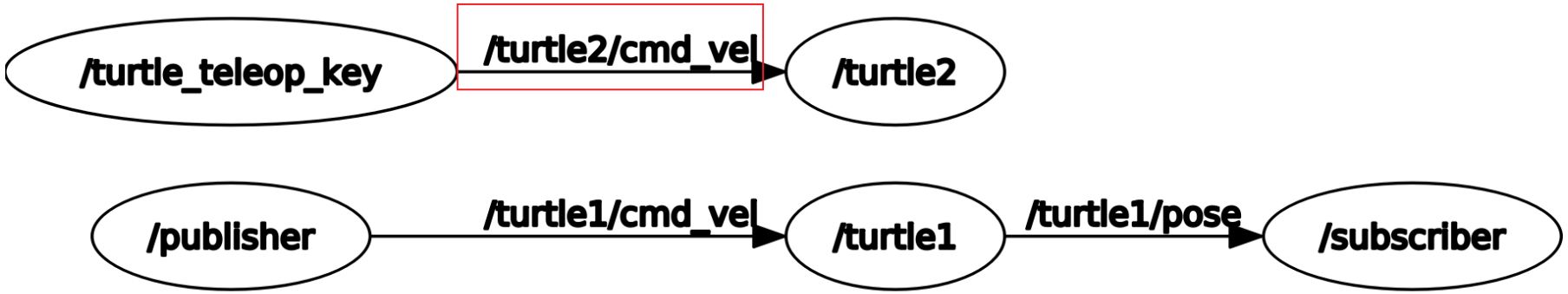
turtlesim_remap_include.launch

```
<launch>
    <include file="$(find learning_launch)/launch/turtlesim_parameter_config.launch" />
    <node pkg="turtlesim" type="turtlesim_node" name="turtle2" output="screen"  >
        <remap from="turtle1" to="turtle2"/>
    </node>
    <node pkg="turtlesim" type="turtle_teleop_key" name="turtle_teleop_key" output="screen">
        <remap from="/turtle1/cmd_vel" to="/turtle2/cmd_vel"/>
    </node>

</launch>
```

2. Run launch file using roslaunch

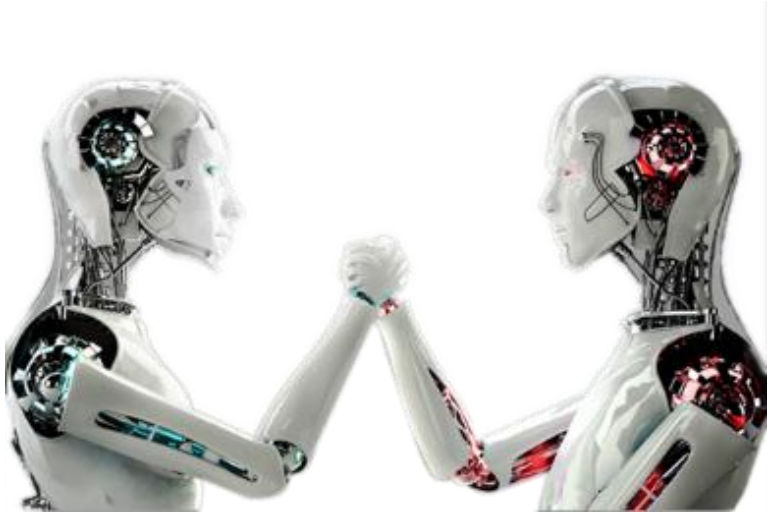
```
$roslaunch learning_launch turtlesim_remap_include.launch
```

3. Result



PART FOUR

Lab Task



Lab Task

1.Complete the lab content of lab ppt ,and submit the zip file of catkin_ws package to BB :

```
catkin_ws
└── src
    ├── beginner_tutorials
    │   ├── include
    │   │   └── beginner_tutorials
    │   ├── msg
    │   ├── scripts
    │   ├── src
    │   └── srv
    ├── lab1_task
    │   ├── include
    │   │   └── lab1_task
    │   └── src
    ├── lab2_task
    │   ├── include
    │   │   └── lab2_task
    │   ├── launch
    │   ├── scripts
    │   ├── src
    │   └── srv
    ├── learning_launch
    │   ├── config
    │   ├── include
    │   │   └── learning_launch
    │   ├── launch
    │   ├── scripts
    │   └── src
    ├── learning_parameter
    │   ├── include
    │   │   └── learning_parameter
    │   ├── scripts
    │   └── src
    └── learning_tf
        ├── include
        │   └── learning_tf
        ├── launch
        ├── scripts
        └── src
```

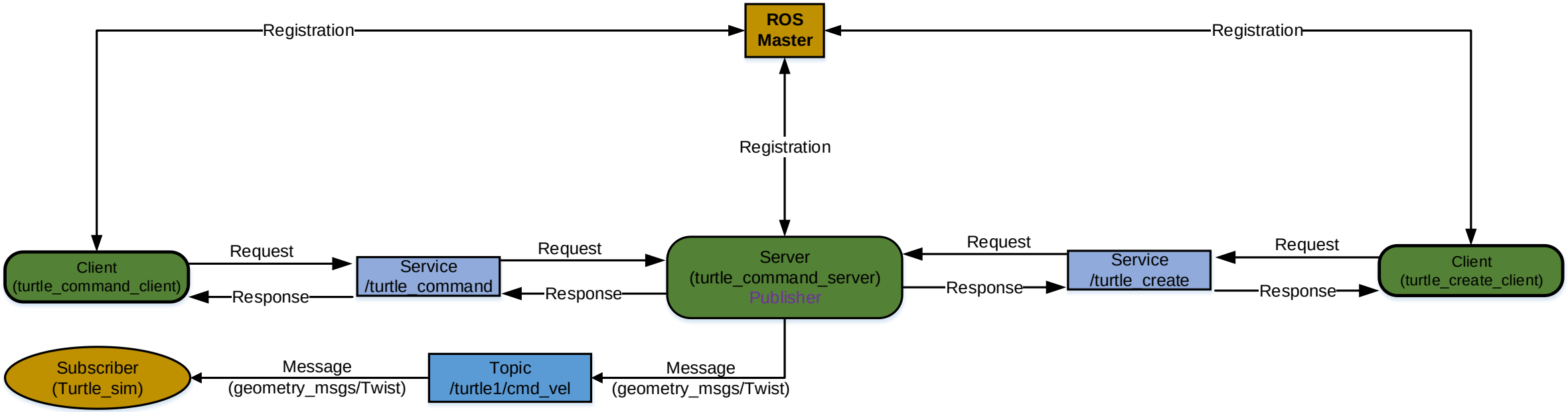
Lab Task

2. (Optional, bonus): The task requires you to do the following

1. A turtle can be generated in the interface by sending the name of a new turtle from the command line, and the position does not overlap.(通过从命令行发送一个新的海龟的名称，可以在海归界面生成新的海龟，并且位置不重叠。)

2. Send instructions from the command line to control the start/stop of any turtle in the interface to make circular movement, and the speed can be controlled by the command line parameters.（从命令行发送指令，控制界面中任意海龟的启动/停止圆周运动，速度可通过命令行参数控制 ）

3.**Complete this task ,and submit the zip file of lab2_task package to BB.**



Lab References

Further References

- **ROS Wiki**
  - http://wiki.ros.org/
- **Installation**
  - http://wiki.ros.org/ROS/Installation
- **Tutorials**
  - http://wiki.ros.org/ROS/Tutorials
- **Available packages**
  - http://www.ros.org/browse/
- **ROS Cheat Sheet**
  - https://github.com/ros/cheatsheet/releases/download/0.0.1/ROScheatsheet_catkin.pdf
- **ROS Best Practices**
  - https://github.com/ethz-asl/ros_best_practices/wiki
- **ROS Package Template**
  - https://github.com/ethz-asl/ros_best_practices/tree/master/ros_package_template

Thanks